EasyOPD: An Easy-to-use On-Policy Distillation Framework for Large Language Models

Jie Sun[1,3,*,§], **Mao Zheng**[2,*], **Mingyang Song**[2,*], **Qiyong Zhong**[1,*,§]
Gengsheng Li[1,*,§], **Zhepei Hong**[1], **Chang Wu**[1], **Pengfei Liu**[3]
Junfeng Fang[4,†], **Xiang Wang**[1,†]
[1] University of Science and Technology of China [2] LLM Department, Tencent
[3] Shanghai Innovation Institute [4] National University of Singapore
https://github.com/lds-ustc/EasyOPD

Abstract

Conventional language-model distillation often relies on fixed teacher-generated data, which may not cover the states encountered by an evolving student policy. On-policy distillation (OPD) instead collects teacher or evaluator supervision on student-generated rollouts. However, existing OPD methods differ substantially in supervision form, tokenizer compatibility, teacher access, and supervision granularity, leading to fragmented implementations that are difficult to reproduce and extend. We present EASYOPD, an on-policy distillation framework built on verl, a distributed reinforcement-learning framework for large language models. EASYOPD separates user-side configuration, method-specific supervision logic, and verl-based execution. Its method modules connect to the shared backend through extension boundaries for loss construction, rollout metadata, reward processing, tokenizer alignment, and teacher-side computation. We implement representative methods covering logit-based OPD, cross-tokenizer OPD, on-policy self-distillation, and step-wise OPD. Experiments on reasoning, code-generation, scientific-knowledge, and tool-use benchmarks show that these implementations can be executed through the same verl-based backend while retaining their method-specific objectives and task-dependent performance profiles. We release EASYOPD with runnable YAML configurations, documentation, an installable demonstration package, and a demonstration video.

1 Introduction

Large language model distillation transfers capabilities from a stronger teacher to a smaller, more efficient student, aiming to reduce deployment costs while retaining reasoning, coding, and instruction-following abilities (Alkhulaifi et al., 2021). Conventional distillation uses teacher responses or token distributions on fixed contexts (Yang et al., 2024b). At inference, however, the student conditions on its own prefixes and may encounter contexts poorly covered by offline supervision (Gu et al., 2024; Ko et al., 2024). On-policy distillation (OPD) mitigates this mismatch through a common loop: the current student generates rollouts, a teacher or evaluator supervises them, and the resulting signals guide student updates (Song and Zheng, 2026; Lu and Thinking Machines Lab, 2025). OPD methods vary by supervision interface, tokenizer compatibility, and granularity, with overlapping settings including logit-based distillation, cross-tokenizer alignment, self-OPD, black-box or rubric-based feedback, and step-wise or agentic supervision.

A shared algorithmic paradigm does not yield a common system interface. Depending on the supervision, OPD may span rollout generation, teacher inference, cross-tokenizer alignment, reward construction, distributed data flow, and optimization. For example, cross-tokenizer OPD requires alignment across teacher and student tokenizations before constructing compatible supervision, whereas black-box or rubric-based OPD may convert textual judgments or scores into rewards; these patterns require different extension points. Existing frameworks address complementary parts of the OPD workflow: TRL (von Werra et al., 2020) provides trainer-centric distillation APIs; verl (Sheng et al., 2024) and slime (Zhu et al., 2025) support scalable on-policy rollout and distributed execution; and KDFlow (Zhang et al., 2026) provides a distillation-oriented stack. However, as summarized in Table 1, we find no single reviewed framework that documents method-local extension boundaries across the heterogeneous supervision interfaces considered here while reusing a common distributed backend. This gap calls for a lightweight OPD abstraction linking method-specific supervision to a shared execution substrate.

* Equal Contribution. † Corresponding Authors.
§ Work done during internship at Tencent.

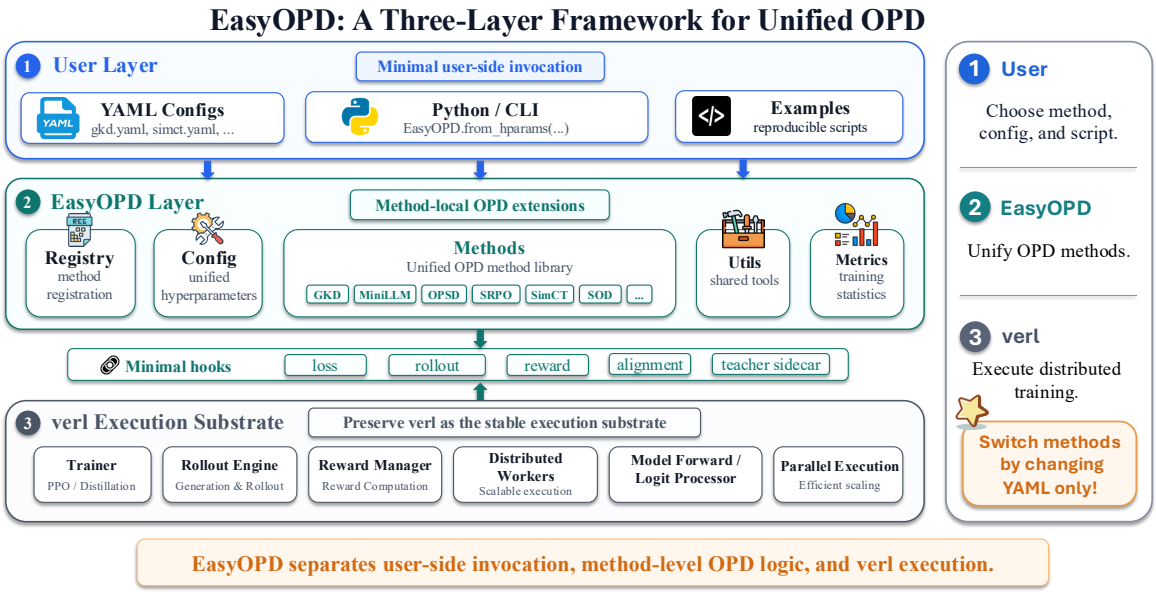

Figure 1: Architecture of EASYOPD. The user layer provides configuration-based invocation, the EASYOPD layer encapsulates method-local OPD logic, and verl provides distributed execution. Hooks for loss, rollout, reward, alignment, and teacher-side computation connect method modules to verl, allowing supported methods to share the same execution backend and be selected via YAML configurations.

To fill this gap, we present EASYOPD, a unified OPD framework built on verl. EASYOPD reuses verl for rollout generation, model execution, and optimization while adding a lightweight abstraction layer for method-local supervision. Method modules connect to the shared training flow through explicit extension points: cross-tokenizer methods localize alignment and supervision construction, whereas black-box or rubric-based methods localize feedback acquisition and training-signal construction. This design turns OPD integration from framework-wide modification into method-local extension.

To realize this design, EASYOPD adopts the three-layer architecture shown in Figure 1. The user layer provides Python and CLI entry points and YAML configurations for methods, models, datasets, and hyperparameters; the EASYOPD layer manages method registration, method-local modules, and supervision diagnostics, connecting to verl through dispatch hooks for loss, rollout metadata, reward, alignment, and teacher-side computation; and the verl layer handles distributed rollout, model execution, worker orchestration, reward computation, and optimization. Users select methods through configuration, while developers add a method by registering a method-local module with the required hooks, so that the core trainer and workers reach method-specific supervision only through the shared dispatch layer.

We instantiate this design with representative methods across three settings: cross-tokenizer OPD, on-policy self-distillation, and step-wise OPD; the framework also implements logit-based OPD, and its extension boundaries are designed to accommodate additional supervision forms such as black-box or rubric-based feedback. Within each setting, the compared methods share the model initialization, training data, execution backend, and evaluation procedure where applicable, while retaining their original supervision designs. We release the code under the Apache License 2.0 on GitHub with reproducible YAML configurations, documentation, and an installable demonstration package with runnable examples.

2 Background and Motivation

Previous Solutions. Knowledge distillation (KD) typically trains a smaller student model to imitate the predictive behavior of a larger or more capable teacher (Hinton et al., 2015; Ko et al., 2024). For autoregressive language models, common offline approaches include sequence-level distillation, in which the student is trained on teacher-generated sequences (Kim and Rush, 2016), and supervised token-level KD, in which the student matches the teacher's next-token distributions along prefixes induced by a fixed offline set of output se-

quences (Agarwal et al., 2024). These output sequences may be ground-truth or teacher-generated. In their purely offline form, both approaches optimize over an output-prefix distribution that does not track the current student's rollout distribution. During autoregressive inference, however, the student conditions on its own previous predictions. A deviation can therefore shift subsequent prefixes away from those observed during offline distillation, creating a training–inference distribution mismatch and allowing errors to compound across decoding steps (Lin et al., 2020; Agarwal et al., 2024).

On-Policy Distillation Mechanism. On-policy distillation (OPD) collects teacher supervision on outputs generated by the current student, rather than relying only on fixed offline sequences. During training, the student generates responses or trajectories, and the teacher provides supervision on the resulting prefixes or states. The student is then updated, and new rollouts are generated with the updated policy. In a common white-box setting, the teacher provides next-token distributions on student-generated prefixes, and the student minimizes a token-level divergence from the teacher (Agarwal et al., 2024). The defining property of OPD is therefore that supervision is collected on states induced by the current student, rather than the use of any particular distillation objective (Lu and Thinking Machines Lab, 2025; Song and Zheng, 2026).

Training on student-generated states helps reduce the mismatch between offline training data and the prefixes encountered during autoregressive inference, while providing guidance on errors made by the student itself (Agarwal et al., 2024; Ko et al., 2024). However, OPD also turns distillation into an iterative pipeline that must coordinate student rollout, teacher inference, supervision construction, and model optimization. Since OPD methods differ in teacher access, feedback form, and training objectives (Song and Zheng, 2026), a practical framework should support this shared pipeline while allowing method-specific supervision to be implemented consistently.

3 Design and Implementation

EasyOPD provides a unified OPD training process on top of verl. Instead of treating each OPD method as an isolated training script, it decomposes an OPD run into four reusable stages: method invocation, configuration resolution, supervision construction, and distributed execution. This section describes how these stages form a unified workflow that connects diverse OPD methods to the same verl execution substrate.

3.1 A Unified OPD Workflow

Although OPD methods differ in their supervision interfaces, they share a common training flow. Given a teacher model, a student model, and training data, the student first generates on-policy rollouts; the selected method then constructs supervision on these rollouts, such as teacher logits, aligned token distributions, judge rewards, or step-level feedback; finally, the training system consumes this supervision through loss computation, reward aggregation, or trajectory-level weighting. EasyOPD makes this shared process explicit and standardizes invocation and dispatch, without requiring all methods to share a single loss, teacher interface, or evaluation metric. As shown in Figure 1, the user layer specifies what to run, the EasyOPD layer defines how supervision is constructed, and the verl layer executes the resulting training process on top of its distributed runtime (Sheng et al., 2024).

3.2 Declarative Invocation and Configuration

EasyOPD exposes OPD methods through declarative configurations and launch scripts. Released methods can be selected through method-specific YAML configurations without manually editing rollout workers, reward managers, logit processors, or trainer-side loss assembly, which turns method selection into a configuration-level operation and makes experiments easier to reproduce. Figure 2 shows a concrete running example in which the same `from_hparams` entry point and a single launch command drive all three studied settings—cross-tokenizer OPD, on-policy self-distillation, and step-wise OPD—together with their baselines, changing only the method name and its YAML configuration.

Different OPD methods require different options: logit-based methods may specify divergence directions or top-k support; cross-tokenizer methods may specify alignment or overlap-vocabulary behavior; rubric-based methods may specify judge prompts or reward aggregation rules; step-wise methods may specify trajectory segmentation or weighting strategies. EasyOPD keeps these choices in a unified configuration space and resolves them through method registration, so new methods can

Table 1: Comparison of OPD supervision interfaces and framework properties. ✓ denotes native or first-class support, ○ denotes partial or experimental support, and × denotes no documented support. *On-policy self-distillation* refers to settings in which the teacher is derived from the student model and supervises student-generated rollouts. The comparison reflects publicly documented functionality as of July 2026, using the versions or commits listed in Appendix A, where per-cell evidence is also provided.

	OPD supervision-interface coverage				**Framework properties**		
Framework	Logit OPD	Cross-tok. OPD	On-policy self-distill.	Step-wise OPD	Released YAML/CLI	Documented method extension API	Compatible w/ multi-node
TRL	✓	○	○	×	✓	×	○
verl	✓	×	×	○	✓	×	✓
slime	✓	×	○	○	✓	×	✓
KDFlow	✓	✓	✓	×	✓	○	○
Ours	✓	✓	✓	✓	✓	✓	✓ (via verl)

One entry point for every OPD setting in EASYOPD

```python
from easyopd import EasyOPD
print(EasyOPD.list_methods())   # [..., 'simct','uld','alm','dskd','sdpo','sod', ...]
# The SAME call selects any method; only the name + YAML change.
m = EasyOPD.from_hparams("simct", config_path="easyopd/config/simct.yaml")  # cross-tokenizer OPD (uld|alm|dskd)
m = EasyOPD.from_hparams("sdpo",  config_path="easyopd/config/sdpo.yaml")   # on-policy self-distillation (vs
    grpo)
m = EasyOPD.from_hparams("sod",   config_path="easyopd/config/sod.yaml")    # step-wise OPD (vs response-level
    opd, grpo)
```

```
# Each method (and its baselines) is one launch script; the config selects the method and the hooks it activates.
$ bash examples/simct/run_simct.sh   # cross-tokenizer: uld|alm|dskd
$ bash examples/sdpo/run_sdpo.sh     # self-distillation: grpo
$ bash examples/sod/run_sod.sh       # step-wise: response-level opd, grpo
```

```
# The registry activates only the hooks a method needs and logs supervision-specific diagnostics:
#   simct -> alignment + teacher sidecar + loss  : simct/xtok_kd_loss, simple/teacher_loss_tokens_mean
#   sdpo  -> reprompt self-teacher + loss        : actor/sdpo/loss, self_distillation/reprompt_sample_fraction
#   sod   -> step-wise KL reweighting            : actor/token_kl, stepwise_opd_coef
```

Figure 2: A running example of EASYOPD. The three studied settings—cross-tokenizer OPD, on-policy self-distillation, and step-wise OPD—and their baselines are all reached through the *same* `from_hparams` entry point or a single launch script, changing only the method name and its YAML config. The registry then activates only the hooks each method needs, verl performs rollout and optimization, and EASYOPD reports supervision-specific diagnostics. All API calls, method names, launch commands, and diagnostic metric names are taken from the released repository.

be added without hard-coding method choices into the trainer.

3.3 Method-Local Supervision Logic

The core of EasyOPD is to localize OPD-specific supervision logic: a method defines how supervision is obtained, transformed, consumed, and diagnosed. Standard OPD may only require a token-level distillation objective, cross-tokenizer OPD may require tokenizer or span alignment and a common supervision space, rubric-based OPD may convert judge feedback into rewards, and step-wise OPD may attach feedback to intermediate reasoning states or multi-turn trajectories. EasyOPD does not force these methods into a single loss-only abstraction; it gives each a local space to implement the supervision interface it needs.

Method-local logic also includes diagnostics. Implemented method modules expose supervision-specific diagnostics in addition to task performance, so users can check whether the intended supervision signal is active, not only the final loss. For example, the released cross-tokenizer implementation reports the overlap-vocabulary size, the number of valid and span-level aligned segments, the count of skipped samples, and the valid-span ratio, while the step-wise implementation reports the mean step-level weight and the number of supervised steps. These metrics help users debug failures that final task performance alone cannot explain.

3.4 Execution through Thin verl Hooks

EasyOPD reuses verl as the execution substrate instead of reimplementing distributed training. verl

provides the system-heavy components required by OPD, including rollout generation, reward computation, model forwarding, logit processing, distributed workers, trainer-side optimization, and parallel execution, and EasyOPD connects method-local supervision logic to this substrate through a small set of thin hooks.

A hook is a dispatch boundary rather than a method implementation: it exposes a stable interaction point in the training pipeline and routes the corresponding data or control flow to the selected method module. The hook set covers the main entry points of OPD supervision: loss hooks for token- or distribution-level supervision, rollout hooks for trajectory metadata, reward hooks for judge or rubric feedback, alignment hooks for token- or span-level mapping, and teacher-sidecar hooks for teacher-side signals such as logits, hidden states, or judge outputs, as used respectively by logit-based, feature-based (Zhang et al., 2024), and rubric-based methods.

This design balances expressiveness and maintainability. Writing every method directly into verl would create method-specific branches, while exposing only a single fixed interface would make alignment-, reward-, or trajectory-based OPD methods difficult to express. Thin hooks keep method logic local while verl remains the shared backend. EasyOPD thus inherits rollout generation, worker orchestration, optimization, and distributed execution from verl; its contribution is the dispatch layer connecting these components to method-local supervision logic.

4 Experiments

4.1 Experimental Setup

We evaluate EASYOPD in three settings that exercise different method-specific interfaces. Cross-tokenizer OPD requires supervision across different teacher and student tokenizations. On-policy self-distillation instantiates teacher and student policies from the same model under different conditioning contexts. Step-wise OPD assigns supervision to intermediate reasoning or tool-interaction steps. These settings cover variation in supervision space, teacher conditioning, and supervision granularity, and are not intended as a single cross-regime leaderboard.

Cross-tokenizer OPD. Standard OPD directly compares teacher and student next-token distributions, which are not naturally aligned under different tokenizers; cross-tokenizer methods address this by constructing a shared supervision space. We compare SimCT (Sun et al., 2026), ULD (Boizard et al., 2025), ALM (Minixhofer et al., 2025), and DSKD (Zhang et al., 2024), distilling Qwen2.5-7B-Instruct (Yang et al., 2024a) into Phi-4-mini-Instruct (Abouelenin et al., 2025). All methods start from the same student checkpoint, warm-started on a shared 10K teacher-generated math and code corpus, and use the same training prompts and optimization budget. We evaluate on MATH500 (Lightman et al., 2024), GSM8K (Cobbe et al., 2021), MBPP (Austin et al., 2021), and LiveCodeBench v6 (Jain et al., 2025), reporting exact-match accuracy for reasoning and pass@1 for code.

On-policy self-distillation. On-policy self-distillation (OPSD) uses the same base model under different conditioning contexts: the student generates a response from the original prompt, while the self-teacher re-evaluates it with additional feedback. We instantiate this setting with SDPO (Hübotter et al., 2026), using Qwen3-8B (Qwen Team, 2025) as both the student and self-teacher, and compare it with the base model and GRPO (Shao et al., 2024). We evaluate Chemistry from SciKnowEval (Feng et al., 2024), Biology from SciKnowEval (Feng et al., 2024), Tool Use from ToolAlpaca (Tang et al., 2023), and mathematical reasoning on GSM8K (Cobbe et al., 2021). We report the highest Average@16 reached within one-hour and five-hour wall-clock budgets, where Average@16 is the mean correctness over 16 independently sampled responses per question.

Step-wise OPD. Response-level OPD (Agarwal et al., 2024) applies teacher supervision uniformly across a trajectory, although supervision on later steps may become less reliable after erroneous tool interactions. SOD (Zhong et al., 2026) instead reweights the distillation loss by step-level student–teacher divergence. We use Qwen3-1.7B (Qwen Team, 2025) as the student and a GRPO-optimized Qwen3-4B (Zhong et al., 2026) as the teacher, comparing the base model, GRPO (Shao et al., 2024), response-level OPD, and SOD under a shared SFT initialization, training data, and optimization budget (Zhong et al., 2026). We evaluate on AIME 2024, AIME 2025 (Mathematical Association of America, 2026), and LiveCodeBench v6 (Jain et al., 2025); each score is Average@32 over 32 sampled responses per problem, and Avg. is their unweighted mean.

Table 2: Cross-tokenizer OPD results. Reasoning scores are exact-match accuracy, code scores are pass@1, and Avg. is their unweighted mean. Best per column in bold.

Method	MATH500	GSM8K	MBPP	LCB-v6	Avg.
Base	54.2	86.0	53.2	7.4	50.2
ULD	57.0	85.1	55.4	9.1	51.7
ALM	58.2	84.9	**56.4**	**12.0**	52.9
DSKD	58.8	86.4	56.2	9.7	52.8
SimCT	**59.6**	**86.7**	**56.4**	11.4	**53.5**

Table 3: SDPO and GRPO in the OPSD setting under one-hour and five-hour wall-clock budgets. Average@16 is the mean correctness over 16 sampled responses per question; Base is the untrained checkpoint, reported once per task. We evaluate every five steps and report the best Average@16 observed within each budget; the best per task and budget is in bold.

Method	avg@16							
	Chemistry		Biology		Tool Use		GSM8K	
	1h	5h	1h	5h	1h	5h	1h	5h
Base	41.1		30.5		57.7		86.1	
GRPO	51.2	72.0	32.1	53.9	62.3	**65.8**	**91.8**	**93.3**
SDPO	**70.4**	**78.7**	**51.3**	**59.9**	**62.8**	62.8	87.7	87.7

Shared implementation. All methods are implemented in EASYOPD and executed through the same verl-based infrastructure (Sheng et al., 2024). Within each setting, we keep the model configuration, training data, execution backend, and evaluation procedure fixed where applicable, while retaining each method's original supervision design. Additional optimization, sampling, hardware, and checkpoint details are provided in the Appendix B.

4.2 Results

Cross-tokenizer OPD. Table 2 shows that all four evaluated cross-tokenizer methods improve the unweighted average over the base student. SimCT obtains the highest average (53.5, +3.3 over the base) and the best MATH500, ALM leads on MBPP and LiveCodeBench v6, and DSKD is best on GSM8K. This confirms that the evaluated cross-tokenizer objectives run through EASYOPD's alignment and loss interfaces under a matched setup, while retaining distinct task-dependent profiles.

On-policy self-distillation. Table 3 compares SDPO and GRPO under the two wall-clock budgets. SDPO's gains are largest on Chemistry and Biology (+19.2 points at one hour, +6.7 and +6.0 at five hours), while GRPO is stronger on GSM8K and on the five-hour Tool Use result. SDPO's relative performance is thus task-dependent in the evaluated setting.

Table 4: Step-wise OPD with Qwen3-1.7B as the student. OPD and SOD use a GRPO-optimized Qwen3-4B teacher. Values are percentages; Avg. is the unweighted mean across AIME 2024, AIME 2025, and LiveCodeBench v6. Best per column in bold.

Method	AIME24	AIME25	LCB-v6	Avg.
Base	17.19	12.81	12.41	14.14
GRPO	34.06	26.25	21.93	27.41
OPD	38.13	36.04	30.27	34.81
SOD	**43.65**	**38.33**	**37.88**	**39.95**

Step-wise OPD. Table 4 shows that response-level OPD exceeds GRPO on all three benchmarks, and SOD obtains the highest observed score on each, for an unweighted average of 39.95 (+5.1 over response-level OPD). These results are consistent with the intended role of step-wise weighting, but do not by themselves establish that every step-wise OPD method will outperform response-level OPD.

Overall findings. Across the three case studies, EASYOPD executes the evaluated methods through the same verl-based backend while preserving their distinct alignment, teacher-conditioning, and supervision-granularity requirements, and the results should be read within each setting rather than as a cross-regime ranking.

5 Conclusion and Future Work

We present EasyOPD, a method-oriented framework for integrating on-policy distillation methods on a shared verl execution backend, separating user-side configuration, method-specific supervision, and distributed execution. Through case studies in cross-tokenizer OPD, on-policy self-distillation, and step-wise OPD, we show that the evaluated methods retain their distinct supervision requirements within one implementation structure via method-local extension boundaries on top of verl. Future work will expand the set of supported methods and supervision interfaces, strengthen the built-in diagnostics, and extend the evaluation to larger models and additional system-level metrics such as throughput and memory footprint.

Limitations

EasyOPD currently depends on verl and has been validated on a representative, rather than exhaustive, set of OPD supervision interfaces. The three experimental settings use different models, tasks, and metrics and should not be interpreted as a single cross-regime leaderboard. Teacher inference, tokenizer alignment, and external evaluation can introduce method-specific costs beyond the framework dispatch overhead. Although the extension boundaries are designed to accommodate additional supervision forms, support should be claimed only after the corresponding implementation and runnable configuration are released. EasyOPD is released under the Apache License 2.0.

References

Microsoft: Abdelrahman Abouelenin, Atabak Ashfaq, Adam Atkinson, Hany Awadalla, Nguyen Bach, Jianmin Bao, Alon Benhaim, Martin Cai, Vishrav Chaudhary, Congcong Chen, Dong Chen, Dongdong Chen, Jun-Kun Chen, Weizhu Chen, Yen-Chun Chen, Yi-ling Chen, Qi Dai, Xiyang Dai, Ruchao Fan, and 55 others. 2025. Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs. *arXiv preprint*, arXiv:2503.01743.

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, Vienna, Austria.

Abdolmaged Alkhulaifi, Fahad Alsahli, and Irfan Ahmad. 2021. Knowledge distillation in deep learning and its applications. *PeerJ Computer Science*, 7:e474.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *arXiv preprint*, arXiv:2108.07732.

Nicolas Boizard, Kevin El Haddad, Céline Hudelot, and Pierre Colombo. 2025. Towards cross-tokenizer distillation: The universal logit distillation loss for LLMs. *Transactions on Machine Learning Research (TMLR)*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint*, arXiv:2110.14168.

Kehua Feng, Keyan Ding, Weijie Wang, Xiang Zhuang, Zeyuan Wang, Ming Qin, Yu Zhao, Jianhua Yao, Qiang Zhang, and Huajun Chen. 2024. SciKnowEval: Evaluating multi-level scientific knowledge of large language models. *arXiv preprint*, arXiv:2406.09098.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2024. MiniLLM: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*.

Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint*, arXiv:1503.02531.

Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. 2026. Reinforcement learning via self-distillation. *arXiv preprint*, arXiv:2601.20802.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*.

Yoon Kim and Alexander M. Rush. 2016. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1317–1327, Austin, Texas. Association for Computational Linguistics.

Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. 2024. DistiLLM: Towards streamlined distillation for large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235, Vienna, Austria.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*.

Alexander Lin, Jeremy Wohlwend, Howard Chen, and Tao Lei. 2020. Autoregressive knowledge distillation through imitation learning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 6121–6133, Online Event.

Kevin Lu and Thinking Machines Lab. 2025. On-policy distillation. *Thinking Machines Lab: Connectionism*. https://thinkingmachines.ai/blog/on-policy-distillation.

Mathematical Association of America. 2026. MAA invitational competitions: American invitational mathematics examination (AIME). https://maa.org/maa-invitational-competitions/. Accessed: 2026-07-01.

Benjamin Minixhofer, Ivan Vulić, and Edoardo Maria Ponti. 2025. Universal cross-tokenizer distillation via approximate likelihood matching. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*.

Qwen Team. 2025. Qwen3 technical report. *arXiv preprint*, arXiv:2505.09388.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint*, arXiv:2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*.

Mingyang Song and Mao Zheng. 2026. A survey of on-policy distillation for large language models. *arXiv preprint*, arXiv:2604.00626.

Jie Sun, Mao Zheng, Mingyang Song, Qiyong Zhong, Yilin Cheng, Bichuan Feng, Pengfei Liu, Junfeng Fang, and Xiang Wang. 2026. Simct: Recovering lost supervision for cross-tokenizer on-policy distillation. arXiv preprint arXiv:2605.07711. *Preprint*, arXiv:2605.07711.

Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. 2023. ToolAlpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint*, arXiv:2306.05301.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. 2020. TRL: Transformers reinforcement learning. `https://github.com/huggingface/trl`. GitHub repository.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 22 others. 2024a. Qwen2.5 technical report. *arXiv preprint*, arXiv:2412.15115.

Chuanpeng Yang, Wang Lu, Yao Zhu, Yidong Wang, Qian Chena, Chenlong Gao, Bingjie Yan, and Yiqiang Chen. 2024b. Survey on knowledge distillation for large language models: Methods, evaluation, and application. *arXiv preprint*, arXiv:2407.01885.

Songming Zhang, Xue Zhang, Zengkui Sun, Yufeng Chen, and Jinan Xu. 2024. Dual-space knowledge distillation for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, pages 18164–18181, Miami, Florida, USA.

Songming Zhang, Xue Zhang, Tong Zhang, Bojie Hu, Yufeng Chen, and Jinan Xu. 2026. KDFlow: A user-friendly and efficient knowledge distillation framework for large language models. *arXiv preprint*, arXiv:2603.01875.

Qiyong Zhong, Mao Zheng, Mingyang Song, Xin Lin, Jie Sun, Houcheng Jiang, Xiang Wang, and Junfeng Fang. 2026. Sod: Step-wise on-policy distillation for small language model agents. *arXiv preprint*, arXiv:2605.07725.

Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. 2025. slime: An llm post-training framework for rl scaling. `https://github.com/THUDM/slime`. GitHub repository. Corresponding author: Xin Lv.

A Framework Comparison Details

Table 5 provides the framework-specific evidence underlying the ratings in Table 1.

B Experimental Setup Details

We summarize the optimization, sampling, and hardware settings for the three case studies. Unless noted otherwise, all settings are taken from the released configuration files and launch scripts.

Software environment. All experiments run on a node with $8\times$ NVIDIA H20 96GB GPUs (driver 535.161.08, CUDA 12.4) under Ubuntu 22.04, using the released `OpenAgentRL` conda environment (Python 3.11, PyTorch 2.6.0+cu124, verl 0.5.0, Ray 2.47.1, Transformers 4.51.1, vLLM 0.8.5.post1, SGLang 0.4.6.post1, FlashAttention 2.7.4.post1, and xFormers 0.0.29.post2). The full pinned dependency list and a one-click installer are released with the code.

Cross-tokenizer OPD. We distill Qwen2.5-7B-Instruct (vocabulary size 151,936) into a Phi-4-mini-Instruct student (3.8B, vocabulary size 200,064) that is first SFT-warm-started on a shared 10K mixed math-and-code corpus derived from the single-turn prompt–answer pairs of the `mixed_math_code_10k` dataset. Training uses GRPO with a cross-tokenizer KL reward, a learning rate of 5×10^{-7} with a 0.05 warmup ratio, a train batch size of 64, one epoch (154 steps over the $\sim$9.9K training prompts), maximum prompt and response lengths of 4096 tokens, and single-sample rollouts at temperature 0.6. All compared methods share this training-prompt set and optimization budget. We save a checkpoint every 20 steps and, after merging, evaluate each checkpoint with an SGLang-based inference server, reporting exact-match accuracy for reasoning (verified with `math_verify`) and pass@1 for code on MATH500, GSM8K, MBPP, and LiveCodeBench v6.

On-policy self-distillation. SDPO uses Qwen3-8B as both the student and the self-teacher, with an EMA self-teacher (update rate 0.05), top-k logit distillation with $k = 100$, and a symmetric KL interpolation ($\alpha = 0.5$). Following the SDPO setup (Hübotter et al., 2026), we evaluate a checkpoint every five training steps and report the best Average@16 observed within the one-hour and five-hour wall-clock budgets, where Average@16 is the mean correctness over 16 independently sampled responses per question.

Step-wise OPD. We use Qwen3-1.7B as the student and a GRPO-optimized Qwen3-4B as the teacher, with a shared SFT initialization, training data, and optimization budget across compared methods. Rollouts are multi-turn tool-interaction trajectories, and SOD applies step-wise adaptive weighting (with numerical-stability offset $\epsilon = 10^{-6}$ and bound offset $\delta = 0.2$) on top of the response-level OPD objective. Each benchmark score is Average@32 over 32 independently sampled responses per problem.

C Released Methods and Hooks

Table 6 lists the released methods used in this work and the dispatch hooks each one registers. Every method generates on-policy rollouts through verl's shared rollout engine; the hooks in the table are the *additional* method-local extension points a method registers on top of this shared pipeline. Because a hook only routes data or control to a method module, a method activates only the hooks it needs: a logit-based objective such as GKD registers a single loss hook, whereas methods that require teacher-side computation or per-step trajectory metadata additionally register a teacher-sidecar or rollout-metadata hook. All other pipeline stages, including rollout generation, worker orchestration, and optimization, are reused from verl unchanged.

Framework	OPD supervision-interface coverage				Framework properties		
	Logit OPD	Cross-tok. OPD	On-policy self-distill.	Step-wise OPD	Released YAML/CLI	Documented method extension API	Compatible w/ multi-node
TRL	✓	○[a]	○[b]	×	✓	×[c]	○[d]
verl	✓	×	×	○[e]	✓	×[f]	✓
slime	✓[g]	×	○[h]	○[e,g]	✓	×[f]	✓
KDFlow	✓	✓	✓[i]	×	✓	○[j]	○[k]
Ours	✓	✓	✓	✓	✓	✓	✓

Table 5: Framework-specific evidence supporting the ratings in Table 1. Ratings reflect publicly documented functionality of the following versions: EasyOPD builds on verl 0.5.0, and we compare against TRL 0.27.0, the slime repository (accessed 2025-06-19), and the KDFlow implementation (Zhang et al., 2026). Where a capability could not be verified from public documentation or code, it is marked as not documented rather than inferred as absent.

[a] Cross-tokenizer support is provided through experimental distillation components rather than the standard GKD path.
[b] TRL provides an experimental SDPO trainer with live, base, and EMA teacher options, but the support is tied to a specialized trainer.
[c] OPD variants are implemented as separate trainer classes rather than composable method-level extension points.
[d] Multi-node trainer execution is available, but a disaggregated asynchronous rollout–teacher–trainer pipeline is not provided.
[e] Agentic or step-wise supervision is expressible through general reward/RLVR and multi-turn infrastructure, but is not exposed as a dedicated OPD interface.
[f] New variants require method-specific logic across internal rollout, reward/advantage, logit-processing, or loss components.
[g] slime provides OPD through its RL reward/advantage pipeline and offers strong agentic and multi-turn rollout infrastructure.
[h] A released slime example uses the original model as the teacher for self-distillation, but does not expose a general live or EMA self-teacher interface.
[i] KDFlow supports student-to-teacher weight synchronization and EMA teacher updates for on-policy self-distillation.
[j] KDFlow provides decoupled algorithm APIs, but they remain tied to its own distillation stack.
[k] KDFlow supports distributed execution, while its documented on-policy training loop is primarily synchronous.

Table 6: Method-local dispatch hooks registered by each released method, beyond the shared rollout–optimization pipeline that all methods reuse from verl. ✓ marks an activated hook; the rollout-metadata hook attaches per-step or trajectory annotations and is distinct from rollout generation itself. Cross-tokenizer alignment in SimCT reuses the shared `simple` teacher sidecar and is applied inside its loss path.

Method	Loss	Rollout meta.	Teacher sidecar	Setting
GKD	✓	×	×	Logit
SimCT	✓	×	✓	Cross-tok.
SDPO	✓	×	✓	Self-distill.
SOD	✓	✓	×	Step-wise